

Forward engineering methods in the context of BIRD

Content

0.	Version control	2
1.	Introduction	2
1.1	Remark on normalisation & denormalisation	3
2.	Forward engineering methods	3
2.1	General considerations regarding forward engineering	4
2.1.1	<i>Treatment of optionality for enumerated and non-enumerated attributes / columns</i>	4
2.1.2	<i>Conserving referential integrity (via validation rules & Null Explanatory Values (NEVs))</i>	4
2.2	Merging entity types into tables	7
2.2.1	<i>Merging entity types into a supertype / subtype</i>	7
2.2.2	<i>Merging entity types connected via relationship types</i>	14
2.2.3	<i>Merging tables with equal surrogate keys</i>	21

0. Version control

Version	Date	Comments
1.0	08/11/2021	Initial draft
1.1	16/12/2022	Incorporation of comments and suggestions for improvement provided by members of the Work Stream on Prototyping (formerly known as Work Stream on Testing) and of the temporary BIRD subgroup on logical data model and input layer (LDM/IL) review.

1. Introduction

The information that is relevant for fulfilling the reporting requirements covered by the BIRD documentation is described in the BIRD Logical Data Model (LDM). This LDM is a highly normalised model that describes the logic of the business domain (i.e., the information that is relevant for fulfilling the reporting requirements), it does not take into account any implementation specific considerations.

The BIRD Input Layer (IL) is intended to act as an implementation model based on the IL design principles. This IL is derived from the LDM via so called forward engineering methods which we define as a combination of denormalisation and additional (validation) rules to ensure “semantic equivalence” between the LDM and the IL. This document’s purpose is to describe these forward engineering methods, their behaviour and implications¹. It also tries to describe the validation rules² that are necessary to ensure that the data stored in the IL is consistent with the definition of the data structures in the LDM.

To support the reader in distinguishing between logical and technical model artefacts we will use the terms entity type and attribute when referring to the LDM and table and column for the implementation model (e.g. IL). We will use the term domain when referring to both models.

Please note that the examples given in this document were adapted for educational purpose and do not represent the exact state of the LDM or IL.

¹ Forward engineering does not include any form of aggregation. Aggregation of data is an aspect that needs to be managed/handled within transformation (of the data from the input to the output).

² Validation rules are currently under development and will be implemented in the future.

1.1 Remark on normalisation & denormalisation

Before describing the forward engineering methods which is a denormalisation process, we believe it important to define what we mean by denormalisation because “for a practice that’s so widely advocated, there seems to be considerable confusion over what denormalisation actually consists of.”³. Fully aligned with the author of this quote we define denormalisation as the inverse of normalisation and therefore the process to decrease a data structure’s normal form to a lower normal form, e.g., from 3rd normal form to 2nd normal form. In order to avoid losing information the documentation of validation rules for the new data structure resulting from denormalisation is required.

As regards the question when a model is sufficiently denormalised we would like to underline that “With normalisation, where there are clear logical reasons for continuing the process until we reach the highest possible normal form. Do we then conclude that with denormalisation we should proceed until we reach the lowest possible normal form? Surely not; yet there are no logical criteria for deciding exactly where the process should stop. In choosing to denormalise, in other words, we’ve backed from a position that does at least have some solid science and logical theory behind it, and replaced it by one that’s purely pragmatic in nature...”⁴.

2. Forward engineering methods

This section describes the different forward engineering methods. Each method is introduced by a description of the operation itself, i.e., what operation is to-be-applied onto the entity types of the LDM to forward engineer tables of the IL, an example and an analysis of the implied consequences for applying this forward engineering method. The first subsection of this chapter describes general aspects about forward engineering which are relevant for all following sections.

Please note that we have amended the existing LDM by adding additional attributes or changing the optionality of attributes to cover relevant cases regarding forward engineering from a conceptual perspective. Wherever there were adjustments made to the LDM we will highlight it in the respective section or example accordingly.

³ See Database Design and Relational Theory, section 8. Denormalization, subsection What does Denormalization Mean? by C.J. Date

⁴ See Database Design and Relational Theory, section 8. Denormalization, subsection What does Denormalization Mean? by C.J. Date

2.1 General considerations regarding forward engineering

2.1.1 Treatment of optionality for enumerated and non-enumerated attributes / columns

Firstly, we would like to highlight the different treatment of enumerated and non-enumerated attributes in the LDM and columns in the IL with respect to their representation of optionality, i.e., if these attributes / columns are considered as optional or not.

Enumerated attributes / columns are attributes / columns where only a defined set of values is allowed, an example would be the Address country which allows only countries listed in ISO 3166. Non-enumerated attributes / columns on the other hand are attributes / columns where the allowed values are not specified via a set of specific values but defined more broadly, e.g., the Balance sheet total which may be any numeric value.

Optionality of an attribute / column specifies if this attribute / column is optional or mandatory.

For non-enumerated attributes / columns optionality is specified on the attribute / column itself, i.e. the attribute / column in the entity type / table is defined to be optional or mandatory. For example, mandatory attributes in the LDM are indicated via a red star next to the attribute itself as illustrated in the following picture.

Organisation	
* Organisation name	String
* Legal entity identifier (LEI)	String
* Institutional sector	Institutional sectors applicable to Companies
* Number of employees	Integer
* Legal form	Legal form code including Not applicable
* Country	ISO Countries including Not applicable
Balance sheet total	Integer
An Organisation is a Legal person which is an organized group of people with a particular purpose.	

Figure 1: Entity Organisation having mandatory (red star) and non-mandatory Attributes

For enumerated attributes / column optionality is specified differently. It is specified via a specific value, i.e. Not applicable, and enumerated attributes / columns are always defined as mandatory (always have a red star next to the attribute itself). Consequently, if the allowed values for an enumerated attribute / column comprises the value Not applicable this attribute / column is defined as optional. Conversely, if the allowed values do not comprise the value Not applicable, this attribute / column is defined to be mandatory.

2.1.2 Conserving referential integrity (via validation rules & Null Explanatory Values (NEVs))

The second aspect that we would like to highlight concerns the conservation of referential integrity, which may be formulated as: the forward engineering methods that are applied to the LDM should conserve the rules which are specified in the LDM itself.

An example of such a rule is the attribute International organisation code which is only applicable to International organisations, like the International Monetary Fund or the European Central Bank. For all other types of Organisations, like Credit institutions or Non-financial institutions the attribute International organisation code does not exist. If we put different types of Organisations into one table which comprises the column International organisation code we need to ensure that only for Organisations which are International organisations this column is populated with an allowed value, while for Organisations which are not International organisations this column must take the value Not applicable. If we wouldn't conserve this referential integrity constraint the result IL would allow to represent Organisations which are not International organisations having an International organisation code, which is clearly a data quality issue which may result in incorrect aggregation and therefore incorrect output figures.

2.1.2.1. Example

The following example will give the reader an overview of the type of validations that are required to ensure referential integrity. It is also the basis for the following section dedicated to Null Explanatory Values (NEVs) & validation rules.

Let's assume we have two entity types that we want to merge into one table holding the following data:

Organisation		
Legal person identifier	Organisation name	Balance sheet total
mandatory	mandatory	optional
{String}	{String}	{Integer}
Apple Inc.	Apple Inc.	57,000,000,000
Grüne Erde	Grüne Erde	NULL
Goldman Sachs Group, Inc.	Goldman Sachs Group, Inc.	12,000,000,000,000
Other company	Other company	NULL
...	...	...

Figure 2: Content of the entity type Organisation

Natural person		
Legal person identifier	First name	Last name
mandatory	mandatory	mandatory
{String}	{String}	{String}
Marie Curie	Marie	Curie
Albert Einstein	Albert	Einstein
Bernhard Riemann	Bernhard	Riemann
...	...	...

Figure 3: Content of the entity type Natural person

Merging this data into one data result in the following Table:

Organisation & Natural person table					
Legal person identifier	Legal person type	Organisation name	Balance sheet total	First name	Last name
mandatory	mandatory	optional	optional	optional	optional
{String}	{Organisation, Natural person}	{String}	{Integer}	{String}	{String}
Apple Inc.	Organisation	Apple Inc.	57,000,000,000	NULL	NULL
Grüne Erde	Organisation	Grüne Erde	NULL	NULL	NULL
Goldman Sachs Group, Inc.	Organisation	Goldman Sachs Group, Inc.	12,000,000,000,000	NULL	NULL
Other company	Organisation	Other company	NULL	NULL	NULL
Marie Curie	Natural person	NULL	NULL	Marie	Curie
Albert Einstein	Natural person	NULL	NULL	Albert	Einstein
Bernhard Riemann	Natural person	NULL	NULL	Bernhard	Riemann
...	...	...	...	...	...

Figure 4: Resulting Organisation & Natural person table

Which requires the following validation rules to ensure consistency with the data illustrated in Figure 2: Content of the entity type Organisation and Figure 3: Content of the entity type Natural person:

- if Legal person type is Organisation
 - the following columns must be NULL: First name, Last name
 - the following columns must not be NULL: Organisation name
- if Legal person type is Natural person
 - the following columns must be NULL: Organisation name, Balance sheet total
 - the following columns must not be NULL: First name, Last name

Please note that without the validation rules we cannot distinguish between NULL values in the column Balance sheet total, i.e., we don't know if it is NULL because the concept does not apply to the subtype which is the case for *Natural person* or NULL because the concept is optional for the subtype which is the case for *Organisations*.

2.1.2.2. Null Explanatory Values (NEVs) & validation rules

As illustrated in the previous example, it is possible to describe necessary validation rules without so called Null Explanatory Values (NEVs), however information, specifically about the optionality of attributes

Organisation & Natural person table									
Legal person identifier	Legal person type	Organisation name	Null explanatory value (Organisation name)	Balance sheet total	Null explanatory value (Balance sheet total)	First name	Null explanatory value (First name)	Last name	Null explanatory value (Last name)
mandatory	mandatory	optional	mandatory	optional	mandatory	optional	mandatory	optional	mandatory
{String}	{Organisation, Natural person}	{String}	{Not NULL because mandatory for this type, NULL because the concept does not apply for this type}	{Integer}	{NULL or not NULL because optional for this type, NULL because the concept does not apply for this type}	{String}	{Not NULL because mandatory for this type, NULL because the concept does not apply for this type}	{String}	{Not NULL because mandatory for this type, NULL because the concept does not apply for this type}
Apple Inc.	Organisation	Apple Inc.	Not NULL because mandatory for this type	57,000,000,000	NULL or not NULL because optional for this type	NULL	NULL because the concept does not apply for this type	NULL	NULL because the concept does not apply for this type
Grüne Erde	Organisation	Grüne Erde	Not NULL because mandatory for this type	NULL	NULL or not NULL because optional for this type	NULL	NULL because the concept does not apply for this type	NULL	NULL because the concept does not apply for this type
Goldman Sachs Group, Inc.	Organisation	Goldman Sachs Group, Inc.	Not NULL because mandatory for this type	12,000,000,000,000	NULL or not NULL because optional for this type	NULL	NULL because the concept does not apply for this type	NULL	NULL because the concept does not apply for this type
Other company	Organisation	Other company	Not NULL because mandatory for this type	NULL	NULL or not NULL because optional for this type	NULL	NULL because the concept does not apply for this type	NULL	NULL because the concept does not apply for this type
Marie Curie	Natural person	NULL	NULL because the concept does not apply for this type	NULL	NULL because the concept does not apply for this type	Marie	Not NULL because mandatory for this type	Curie	Not NULL because mandatory for this type
Albert Einstein	Natural person	NULL	NULL because the concept does not apply for this type	NULL	NULL because the concept does not apply for this type	Albert	Not NULL because mandatory for this type	Einstein	Not NULL because mandatory for this type
Bernhard Riemann	Natural person	NULL	NULL because the concept does not apply for this type	NULL	NULL because the concept does not apply for this type	Bernhard	Not NULL because mandatory for this type	Riemann	Not NULL because mandatory for this type
...	...	...	...	...	...	...	...	...	...

associated with certain subtypes is only implicitly comprised in the validation rules and rather difficult to extract, i.e. translating data from the IL to the LDM is not feasible with such an approach. Additional NEVs associated with specific columns would make some of the information more explicit and therefore provide additional context as illustrated in the following figure:

Figure 5: Resulting Organisation & Natural person table including Null Explanatory Values (NEVs)

Because of the additional information provided in the Null explanatory value (Balance sheet total) we made it explicit that for Organisations it is NULL or not NULL because optional for this type and for Natural persons the value must be NULL because the concept does not apply for this type (see highlighted in green).

2.2 Merging entity types into tables

2.2.1 Merging entity types into a supertype / subtype

2.2.1.1. Description of the forward engineering method

This method is used to merge multiple entity types which are connected via subtyping into one of their supertypes. The resulting table comprises the distinct union of all attributes of the merged subtypes as columns, the allowed values of the columns of this table result from the union of allowed values of the attributes. If an attribute is only present in some subtypes but not in others it becomes optional in the resulting table.

2.2.1.2. Legal person example

To illustrate the implications of this forward engineering method we will introduce it based on the following example, which was extracted from the LDM and slightly modified to cover additional use cases⁵.

⁵ Here is a list of the amendments we've applied for this example: (1) moved the Institutional sector to the subtype level, (2) added Number of employees, Country and Balance sheet on the Organisation level, (3) added Gender, Country, Nationality, Social security number, Number of employees on the Natural person level.

On the logical level, we distinguish between two (sub-)types of Legal persons, i.e. Organisations and Natural persons, as illustrated in the following picture.

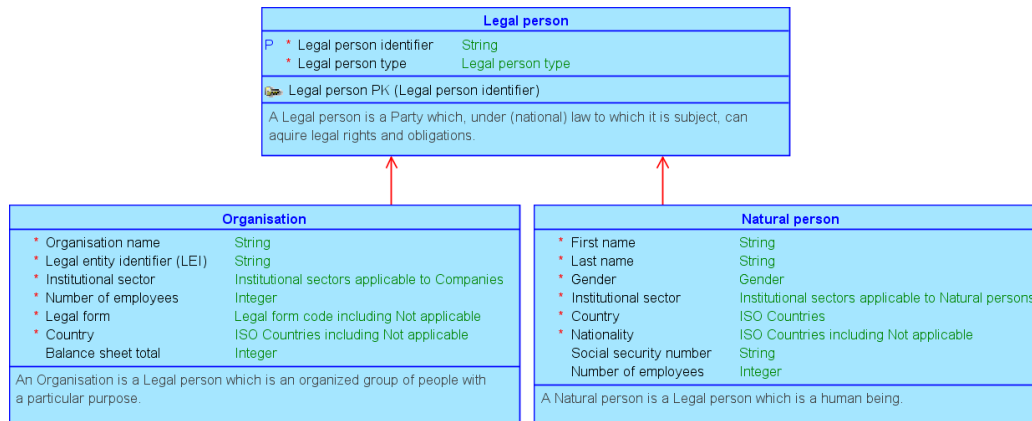


Figure 6: A Legal person is either an Organisation or a Natural person

Please note that this example comprises, among other aspects:

- Enumerated and non-enumerated attributes which are present in some subtypes but not in others, optional and mandatory, e.g. Organisation name, Balance sheet total, Country
- Enumerated attributes which are present in all subtypes, mandatory in all subtypes, e.g. Institutional sector
- Non-Enumerated attributes which are present in all subtypes, mandatory in some, optional in others, e.g. Number of employees

Applying forward engineering methods, specifically Merging entity types into a supertype / subtype, we will end up with one resulting table and additional validation rules.

For educational purposes we will explain the required validation rules based on “data”, therefore let’s assume the entity types are populated as following. The content of the entity type Legal person may hold the following data:

Legal person	
Legal person identifier	Legal person type
mandatory	mandatory
{String}	{Organisation, Natural person}
Apple Inc.	Organisation
Grüne Erde	Organisation
Goldman Sachs Group, Inc.	Organisation
Other company	Organisation
Marie Curie	Natural person
Albert Einstein	Natural person
Bernhard Riemann	Natural person
...	...

Figure 7: Content of the entity type Legal person

Please note that green highlighted columns indicate components of the primary key, in this example it indicates that the value of the column Legal person identifier must be unique. The entity type *Organisation* may be populated as following:

Organisation							
Legal person identifier	Organisation name	Legal entity identifier (LEI)	Institutional sector	Number of employees	Legal form	Country	Balance sheet total
mandatory	mandatory	mandatory	mandatory	mandatory	mandatory	mandatory	optional
{String}	{String}	{String}	{Institutional sectors applicable to Companies}	{Integer}	{Legal form codes including Not applicable}	{ISO Countries including Not applicable}	{Integer}
Apple Inc.	Apple Inc.	Apple LEI	Non-financial corporations	147000	Stock company	United States of America	57,000,000,000
Grüne Erde	Grüne Erde	Grüne Erde LEI	Non-financial corporations	500	Limited	Austria	NULL
Goldman Sachs Group, Inc.	Goldman Sachs Group, Inc.	Goldman Sachs LEI	Credit institution	40500	Stock company	United States of America	12,000,000,000,000
Other company	Other company	Other company LEI	Non-financial corporations	3	Limited	Not applicable	NULL
...	...	...	...	...	...	...	...

Figure 8: Content of the entity type Organisation

And, the entity type *Natural person* may have the following content:

Natural person								
Legal person identifier	First name	Last name	Gender	Institutional sector	Country	Nationality	Social security number	Number of employees
mandatory	mandatory	mandatory	mandatory	mandatory	mandatory	mandatory	optional	optional
{String}	{String}	{String}	{Gender}	{Institutional sectors applicable to Natural persons}	{ISO Countries}	{ISO Countries}	{String}	{Integer}
Marie Curie	Marie	Curie	Female	Households	France	Polish	NULL	2
Albert Einstein	Albert	Einstein	Male	Households	United States of America	German	ssnAlbertEinstein	NULL
Bernhard Riemann	Bernhard	Riemann	Male	Households	Germany	German	NULL	NULL
...	...	...	...	...	...	...	...	...

Figure 9: Content of the entity type Natural person

Wrapping up these entity types will result in one table comprising all the columns illustrated in figures above, we may illustrate it as following:

Legal person Table	
* Legal person identifier	String
* Legal person type	Legal person type
* Institutional sector	Institutional sector applicable to Organisations or Natural persons
* Legal form	Legal form code including Not applicable
* Country	Legal form code including Not applicable
* Gender	Gender including Not applicable
* Nationality	ISO Countries including Not applicable
Organisation name	String
Legal entity identifier (LEI)	String
Number of employees	Integer
Balance sheet total	Integer
First name	String
Last name	String
Social security number	String
Legal person Table PK (Legal person identifier)	

Figure 10: Legal person table

The associated data compatible with the data illustrated in Figure 7: Content of the entity type Legal person to Figure 9: Content of the entity type Natural person is as following:

Figure 11: Content of the Legal person table

Legal person Table													
Legal person identifier	Legal person type	Organisation name	Legal entity identifier (LEI)	Institutional sector	Number of employees	Legal form	Country	Balance sheet total	First name	Last name	Gender	Nationality	Social security number
mandatory	mandatory	optional	optional	mandatory	optional	mandatory	mandatory	optional	optional	optional	mandatory	mandatory	optional
{String}	{Organisation, Natural person}	{String}	{String}	{Institutional sectors applicable to Companies or Natural persons}	{Integer}	{Legal form codes including Not applicable}	{ISO Countries including Not applicable}	{Integer}	{String}	{String}	{Gender including Not applicable}	{ISO Countries including Not applicable}	{String}
Apple Inc.	Organisation	Apple Inc.	Apple LEI	Non-financial corporations	147000	Stock company	United States of America	57,000,000,000	NULL	NULL	Not applicable	Not applicable	NULL
Grüne Erde	Organisation	Grüne Erde	Grüne Erde LEI	Non-financial corporations	500	Limited	Austria	NULL	NULL	NULL	Not applicable	Not applicable	NULL
Goldman Sachs Group, Inc.	Organisation	Goldman Sachs Group, Inc.	Goldman Sachs LEI	Credit institution	40500	Stock company	United States of America	12,000,000,000,000	NULL	NULL	Not applicable	Not applicable	NULL
Other company	Organisation	Other company	Other company LEI	Non-financial corporations	3	Limited	Not applicable	NULL	NULL	NULL	Not applicable	Not applicable	NULL
Marie Curie	Natural person	NULL	NULL	Households	2	Not applicable	France	NULL	Marie	Curie	Female	Polish	NULL
Albert Einstein	Natural person	NULL	NULL	Households	NULL	Not applicable	United States of America	NULL	Albert	Einstein	Male	German	ssnAlbertEinstein
Bernhard Riemann	Natural person	NULL	NULL	Households	NULL	Not applicable	Germany	NULL	Bernhard	Riemann	Male	German	NULL
...	...	...	...	...	...	...	...	...	...	...	...	...	...

Please note that default values are illustrated in red, e.g. the value NULL in the column Organisation name for Marie Curie. The associated validation rules may be formulated as following:

- If the Legal person type is *Organisation*
 - The following columns must be NULL: First name, Last name, Social security number
 - The following columns must not be NULL: Organisation name, Legal entity identifier (LEI), Number of employees
 - The following columns must take the value *Not applicable*: Gender, Nationality
 - The following columns must take specific values:
 - The column *Institutional sector* must take a value from {Institutional sectors applicable to Companies}
- If the Legal person type is *Natural person*
 - The following columns must be NULL: Organisation name, Legal entity identifier (LEI), Balance sheet total
 - The following columns must not be NULL: First name, Last name
 - The following columns must take the value Not applicable: Legal form
 - The following columns must take specific values:
 - The column *Institutional sector* must take a value from {Institutional sectors applicable to Natural persons}

- The column Country must take a value from {ISO Countries}
- The column Gender must take a value from {Gender}
- The column Nationality must take a value from {ISO Countries}

We would like to highlight that we are not able to distinguish between NULL because an attribute was optional for a specific subtype and NULL because the attribute does not apply for a specific subtype. For example, we are not able to distinguish between the NULL values for the column Balance sheet total for the records of Organisation Grüne Erde or Marie Curie, where the latter is NULL because the concept (of Balance sheet total) does not apply for a specific subtype (i.e. for Natural persons) and the former is NULL because the concept is optional for a specific subtype. In theory this information may be extracted from the validation rule stating that the Column Balance sheet total must be NULL for Natural persons, however this requires the creation of logic from statements which is a rather difficult task and consequently not recommended.

2.2.1.3. Subtypes, merging & Relationships

Subtypes may be connected to other entity types via relationship types. When a subtype is merged into its supertype these relationship types need to be managed as well, otherwise we would lose them and therefore our forward engineered model would be incomplete. An example of such a relationship type is the relationship type between an Organisation and its Organisational units, i.e. an Organisation has zero, one-or-many Organisational units. In more practical terms, a Credit institution may have multiple Branches. The situation, as described in the LDM, is illustrated in the following figure.

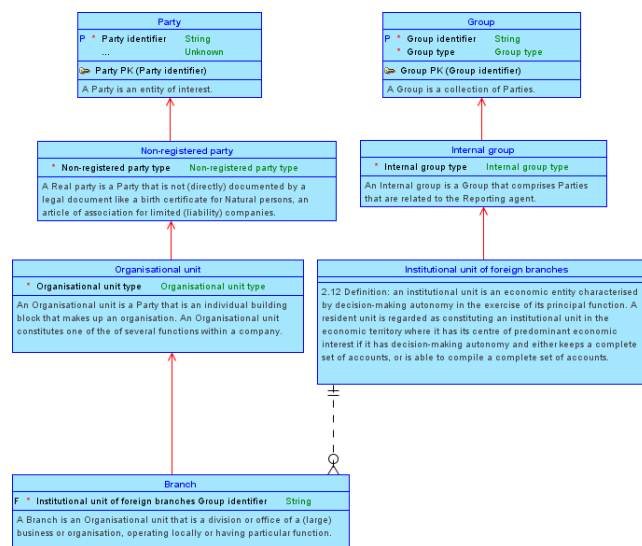


Figure 12: Relationship type between Institutional unit of foreign branches and Organisational unit and their hierarchy

When merging the involved entity types into their respective supertypes, relationship types applicable only to subtypes vanish because they cannot be expressed via SQL foreign key constraints⁶ anymore. Consequently, they cannot be established on the database level directly but have to be validated via validation rules. The following figure illustrates the resulting situation:

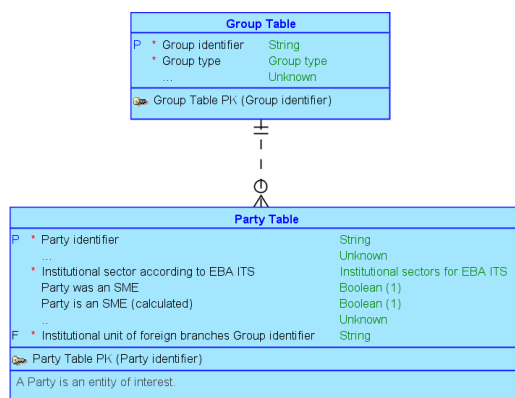


Figure 13: Party & Group table

The following validation rules complete the picture, i.e., ensure that the columns establishing the connection between the tables can only be populated according to the description in the LDM:

- As regards the *Party table*
 - The *Institutional unit of foreign branches Group identifier* must be NULL for all Parties where the *Organisational unit type* is different to *Branch*
 - If the *Institutional unit of foreign branches Group identifier* is not NULL, the following conditions must hold:
 - The *Organisational unit type* must be *Branch*
 - The *Institutional unit of foreign branches Group identifier* must refer to a *Group* where the *Internal group type* is *Institutional unit of foreign branches*

2.2.1.4. Disjoint subtyping

Disjoint subtyping describes a situation where a supertype is split into two different disjoint classifications. In the LDM this is the case for Securities in the form of debt or direct ownership, where one type of subtyping is by classification into Debt security, Equity security and Fund security while the other type of subtyping is by type of identifier into ISIN securities and Non-ISIN securities.

⁶ For additional information regarding foreign key constraints, please see https://www.w3schools.com/sql/sql_foreignkey.asp

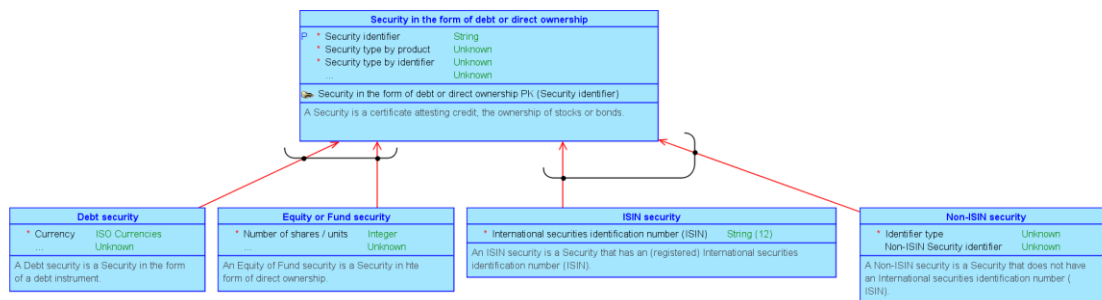


Figure 14: Disjoint subtyping of Security (in the form of debt or direct ownership) by product (Debt security and Equity or Fund security) and identifier (ISIN vs. Non-ISIN)

As regards necessary validation rules to ensure consistency with the LDM the only real difference is that each type of subtyping will result in a separate set of validation rules. In our specific example mentioned above this would result in validation rules for subtyping by classification and validation rules for subtyping by type of identifier. Other than that, the resulting validation rules are similar to the validation rules described in the previous section, see Legal person example.

2.2.1.5. Implications

As discussed in the previous sections the application of this forward engineering method has implications if consistency with the LDM must be ensured.

As regards the columns of the resulting table with respect to the attributes they originate from:

- For non-enumerated columns it needs to be ensured that NULL values are allowed / not allowed according to the specification in the originating entity type, e.g., if a non-enumerated attribute is mandatorily present for a specific subtype, the corresponding column must be different than NULL
- For enumerated columns it needs to be ensured that the value is Not applicable by default according to the specification in the merged entity types, e.g., if an enumerated attribute is not present in all subtypes that are to-be-merged, the resulting column must take the value Not applicable for those subtypes
- For enumerated columns it needs to be ensured that the allowed values correspond to the specification in the LDM with respect to the specific subtypes, e.g., if a mandatory, enumerated attribute allowed only ISO country codes the corresponding column must only allow these values for the specific subtype as well
- For columns establishing a connection with other tables resulting from relationship types of subtypes, it needs to be ensured that the allowed value refers to the allowed subtypes only

Another implication of merging entity types with different attributes is that the resulting table will comprise mainly optional columns (or columns where the value Not applicable is valid).

2.2.2 Merging entity types connected via relationship types

2.2.2.1. Description of the forward engineering method

Merging entity types connected via relationship types is a forward engineering method where two entity types that are connected via a relationship type are merged into one table. This method is intended for identifying relationship types only. The cardinality of the relationship type may be of type one-to-one or one-to-many which are either optional or mandatory. The resulting table's primary key will be equal to the largest primary key of the involved entity types, e.g., if a one-to-many relationship type is involved the resulting table's primary key will be equal to the entity type with the many cardinality.

2.2.2.2. Merging one-to-one, mandatory entity types

Since one-to-one, mandatory relationship types represent an arbitrary split of an entity type into two entity types, at least from a logical perspective, the resulting table's columns will be the result of the union of all attributes of the involved entity types. The optionality of the columns doesn't change.

In the LDM such a one-to-one relationship type is established via *Party* and *Party derived data* where the later entity type comprises derived data, e.g., the *Institutional sector according to EBA ITS*. The LDM situation may be illustrated as following:

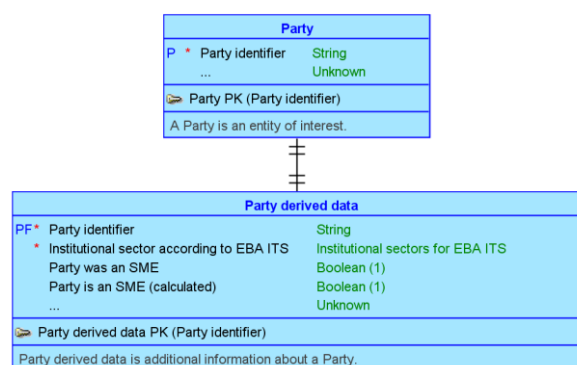


Figure 15: Party & Party derived data

While the resulting table may be illustrated like this:

Party Table		
P *	Party identifier	String
...		Unknown
*	Institutional sector according to EBA ITS	Institutional sectors for EBA ITS
	Party was an SME	Boolean (1)
	Party is an SME (calculated)	Boolean (1)
..		Unknown
Party Table PK (Party identifier)		
A Party is an entity of interest.		

Figure 16: Resulting Party table (indicated)

2.2.2.3. Merging one-to-one, optional entity types ("de-facto subtypes")

When merging de-facto subtypes, all attributes of the optional entity type become optional. Because of the lack of discriminators for de-facto subtypes in the LDM it is not possible to distinguish between optional and mandatory columns in the resulting Table directly. Consequently, validating data of the IL can only be achieved indirectly, i.e., if one of the columns resulting from a mandatory attribute is different to NULL / *Not applicable* all other columns resulting from mandatory attributes must be different to NULL / Not applicable.

2.2.2.4. Merging one-to-many, mandatory

Merging entity types which are connected via a one-to-many, mandatory relationship type creates a table having the primary key of the entity type, which is at the many end of the relationship type, i.e., the one with more attributes contributing to the primary key. Merging such entity types requires validation rules to ensure that duplicated values are consistent. As an example, we look at the situation of *Protection items* and their *Protection values* in the LDM.

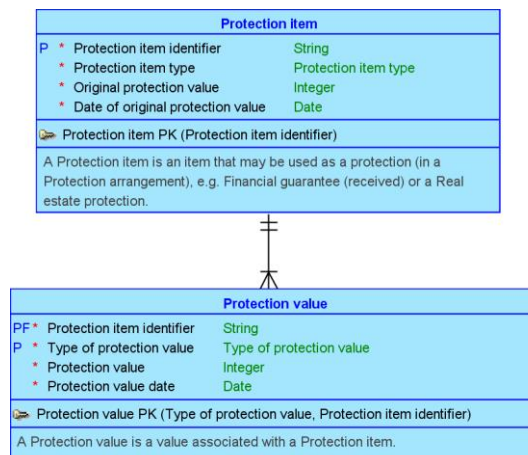


Figure 17: Protection item has one-or-many Protection value(s)

A *Protection item* has one-or-many *Protection values*, while a *Protection value* always belongs to a *Protection item*. The resulting table after merging these entities looks as following:

Protection item & Protection value Table		
P	* Protection item identifier	String
P	* Type of protection value	Type of protection value
	* Protection item type	Protection item type
	* Original protection value	Integer
	* Date of original protection value	Date
	* Protection value	Integer
	* Protection value date	Date
🔑 Protection item & Protection value Table PK (Type of protection value, Protection item identifier)		
A Protection item is an item that may be used as a protection (in a Protection arrangement), e.g. Financial guarantee (received) or a Real estate protection.		

Figure 18: Resulting Protection item & Protection value table

We also need to ensure that the same Protection item always has equal values in the attributes present in the entity type Protection item, otherwise referential integrity would be violated, therefore:

- For each value of the attribute *Protection item identifier* the values of the following attributes need to take the same value: *Protection item type*, *Original protection value*, *Date of original protection value*

2.2.2.5. Merging one-to-many, optional

The situation for entity types connected via a one-to-many, optional relationship type is a little different than the previous situation. The main reason is that with such a relationship type the entity type at the many end of the relationship type is optional. Therefore, when merging such entity types, we need to ensure that the primary key (constraint) is not violated.

For educational purposes we quote the definition of the primary key according to w3schools: "The primary key constraint uniquely identifies each record in a table. Primary keys must contain unique values, and cannot contain NULL values." ⁷ The important aspect that we need to consider in this context is that the values (of the primary key) must not be NULL.

2.2.2.5.1. Entity types on the many side of the relationship with non-enumerated attributes being components of the primary key

Clearly, if we merge two entity types which are related via a one-to-many, optional relationship type, where the primary key of the entity type on the many side of the relationship type comprises non-enumerated attributes which are not present in the other entity type we will violate this definition and therefore the resulting table will not be a valid table in a relational data model. To illustrate the content of the previous paragraph please consider the following situation:

⁷ See https://www.w3schools.com/sql/sql_primarykey.ASP

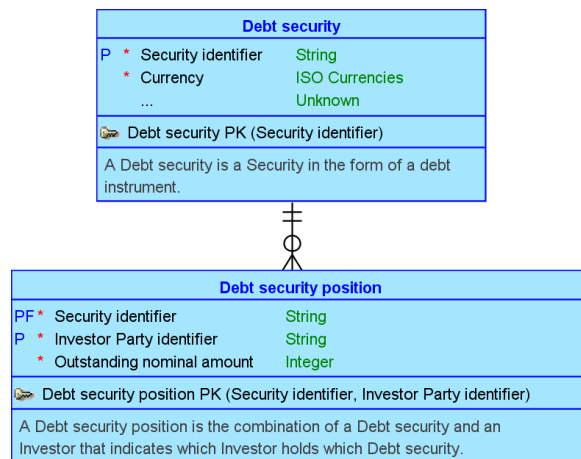


Figure 19: Debt security is involved in zero, one-or-many Debt security position(s)

A Debt security may be held by multiple Investors while an Investor may invest in multiple Debt securities. We call this combination of a Debt security and an Investor a Debt security position. Please note that not every Debt security is involved in a Debt security position, i.e., the relationship type between Debt security and Debt security position is optional. To illustrate the problem, let's consider the following example: in the entity type *Debt security* we register the following security information:

Debt security		
Security identifier	Currency	...
{String}	{Euro, United States Dollar,...}	...
Austrian bond	Euro	...
German bond	Euro	...
Company bond	United States Dollar	...
...	...	...

Figure 20: Registered debt securities

While the Austrian and the German bond are held by Parties, the Company bond is not, therefore the content of the entity type Debt security position might be illustrated as following:

Debt security position		
Security identifier	Investor Party identifier	Outstanding nominal amount
{String}	{String}	{Integer}
Austrian Bond	Some Austrian bank	13
Austrian Bond	Some German bank	19
German Bond	Some Italian bank	23
...	...	...

Figure 21: Content of the entity type Debt security position

If we'd merge the two entity types the resulting table's primary key would be equal to the primary key of the Debt security position entity type. The resulting data for this example would look as following:

Debt security & Debt security position Table			
Security identifier	Investor Party identifier	Currency	Outstanding nominal amount
{String}	{String}	{Euro, United States Dollar,...}	{Integer}
Austrian Bond	Some Austrian bank	Euro	13
Austrian Bond	Some German bank	Euro	19
German Bond	Some Italian bank	Euro	23
Company bond	NULL	United States Dollar	NULL
...	...	...	...

Figure 22: Content of the Debt security & Debt security position table

Since the Company bond is not held by any *Investor*, the Attribute *Investor Party identifier* would be NULL (highlighted in red). This, however, would violate the definition of the primary key and therefore merging entity types connected via a one-to-many, optional relationship type in case the entity type on the many side of the relationship type has a non-enumerated attribute, that is not part of the other entity type's primary key, as a component of the primary key is not possible.

To ensure that duplicated values are consistent we also need to implement additional validation rules, e.g., to ensure that the same value of the attribute *Security identifier* has the same value in the attribute *Currency*.

2.2.2.5.2. Entity types on the many side of the relationship type with enumerated attributes being components of the primary key

The situation is slightly different if the attributes contributing to the primary key of the entity type (on the many side of the relationship type), that are not comprised in the other entity type's primary key, are enumerated. In this case someone might argue that the value *Not applicable* is different to NULL and therefore a valid value for absent data. Consequently, following this argumentation, we would not violate the definition of the primary key. Please note that this argument is debatable because indeed the value *Not applicable* has the same meaning as NULL for non-enumerated attributes.

This constellation occurs in multiple locations in the LDM, for example when applying the role concept to specific entity types. To illustrate the indicated situation and the implications of the application of this forward engineering method in more detail let's look at the role concept applied to *Parties*.

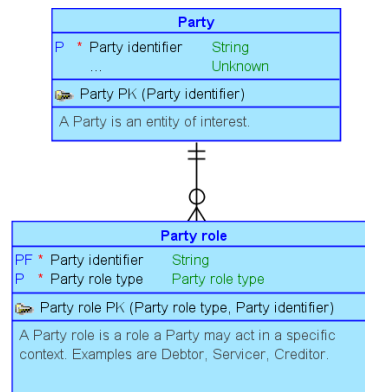


Figure 23: A Party acts in zero, one-or-many Party role(s)

Via this optional, one-to-many relationship type between *Party* and *Party role* a *Party* can act in multiple *Party roles*, e.g. *Debtor*, *Creditor*, *Investor*, at the same time. Like the previous example we will analyse some “data”, e.g.

Party		
Party identifier	Country	...
{String}	{Austria, Germany, ...}	...
Austrian bank	Austria	...
German bank	Germany	...
Some Company	Austria	...
...	...	...

Figure 24: Content of the entity type Party

Some of these *Parties* may act in specific roles, e.g., the Austrian bank may act in the *Party roles Creditor* and *Investor*, the German bank may act in the *Party role Investor*. Please note that *Some Company* does not act in any *Party role*.

Party role	
Party identifier	Party role type
{String}	{Debtor, Creditor, ...}
Austrian bank	Creditor
Austrian bank	Investor
German bank	Investor
...	...

Figure 25: Content of the entity type *Party role*

If we would merge these two entity types, the resulting table's primary key is equal to the primary key of the entity type *Party role*. For the resulting table, the data indicated in Figure 24: Content of the entity type *Party* and Figure 25: Content of the entity type *Party role* would look as following:

Party & Party role Table			
Party identifier	Party role type	Country	...
{String}	{Debtor, Creditor, Investor,...}	{Austria, Germany,...}	...
Austrian bank	Creditor	Austria	...
Austrian bank	Investor	Austria	...
German bank	Investor	Germany	...
Some Company	Not applicable	Austria	...
...	...	...	...

Figure 26: Resulting *Party & Party role* table

Because *Some Company* does not act in any *Party role*, we would use the value *Not applicable* as a default value.

As regards validation rules, similar to the previous example we will need to ensure that every value of *Party identifier* has consistent values for columns arising solely from the entity type *Party*, i.e. ensure consistency of duplicated values.

2.2.2.6. Implications

The main restriction when applying this forward engineering method is that entity types which are connected via one-to-many, optional relationship types, where the entity type on the many side of the relationship type has non-enumerated attributes (which are not present in the other entity type) as components of the primary key, cannot be merged. In case of enumerated attributes contributing to the primary key we may apply this forward engineering method, however the difference between NULL and Not applicable is debateable.

Another important remark from the previous sections is that the resulting tables from entity types connected via one-to-one, optional relationship types ("de-facto subtypes") can only be validated indirectly against the LDM.

Lastly, when applying this method to the entity types connected via one-to-many relationship types we need to specify validation rules to ensure that duplicated values are consistent according to their original primary keys.

2.2.3 Merging tables with equal surrogate keys

2.2.3.1. Description of the forward engineering method

In some situations, we may want to merge entity types having different primary keys. Because of the different primary keys we cannot simply merge these entity types as described in section Merging entity types into a supertype / subtype but we would have to create a matching primary key first. We refer to this operation as the Creation of surrogate keys. It goes without saying that we need additional validation rules to ensure referential integrity if we create such a new surrogate key. After different entity types have the same surrogate key we may want to merge them similar to the situation described in Merging entity types into a supertype / subtype.

2.2.3.2. Creation of surrogate keys

In practical terms, the creation of a surrogate key for an entity type involves the following steps:

- Definition of a new surrogate key, i.e. adding an attribute / column with unique values
- Removing the previous primary key (in terms of key constraints)
- Specifying the new surrogate key to be the primary key
- Specifying validation rules ensuring consistency as regards the previous primary key

2.2.3.3. Merging tables having equal surrogate keys

After creating surrogate keys for different entity types, we may merge these entity types. One prerequisite for merging entity types having the same surrogate key is that the values of the surrogate keys of the entities do not overlap. If these values would overlap, we cannot distinguish between the instances anymore. The required validation rules for this method are similar to the validation rules described in the Legal person example.

2.2.3.4. Implications

The main constraint when applying this forward engineering method is to ensure that the surrogate keys of the different entity types to-be-merged must have non-overlapping values. However, this is a data related topic which can only be described from a meta data perspective like the BIRD documentation.

Another consideration in this context that should be taken into account is the data lineage capabilities between the LDM and the IL, i.e. how “easy” it is to link the entity types of the LDM to the table of the IL or vice-versa. Applying this forward engineering method on too many entity types of the LDM to merge them into one table of the IL will make it more difficult to understand how the tables are related (in detail) with the entity types.